

MoMo SMS Analytics

REST API -- Implementation, Security & DSA Report

Course: Data Structures & Algorithms / API Development
Institution: African Leadership University
Date: May 30, 2026
Team Lead: Gary Murasira
Members: David - Espoir - Merci - Gary

1. Introduction to API Security

Modern REST APIs expose sensitive data over the internet, making security a foundational requirement rather than an afterthought. An unsecured API allows any client to read, modify, or delete records without restriction. Authentication verifies that a caller is who they claim to be, while authorisation controls what that caller is permitted to do.

This project secures a MoMo SMS transaction API using HTTP Basic Authentication -- a simple scheme that demonstrates the core request-response security model. While Basic Auth is appropriate for a learning context, it has well-documented weaknesses that motivate stronger alternatives such as JSON Web Tokens (JWT) and OAuth2, discussed in Section 4.

The API was built on Python's built-in `http.server` module with no external frameworks, giving full visibility into how HTTP authentication headers are parsed and validated at the protocol level. The dataset consists of 1,691 real MoMo SMS records parsed from `modified_sms_v2.xml` and exposed through five CRUD endpoints.

2. API Endpoint Documentation

Base URL: `http://localhost:8000`

Authentication: All endpoints require HTTP Basic Authentication.

Credentials: `admin / password123`

Header: `Authorization: Basic <base64(username:password)>`

Method	Endpoint	Description	Success	Error
GET	<code>/transactions</code>	List all 1,691 transactions	200	401
GET	<code>/transactions/{id}</code>	Fetch a single transaction by ID	200/404	401
POST	<code>/transactions</code>	Create a new transaction	201	400/401
PUT	<code>/transactions/{id}</code>	Partially update an existing transaction	200/404	400/401
DELETE	<code>/transactions/{id}</code>	Delete a transaction by ID	204/404	401

2.1 GET /transactions

Returns a JSON array of all transactions.

Request:

```
curl -u admin:password123 http://localhost:8000/transactions
```

Response -- 200 OK:

```
[{"id": 1, "type": "Incoming Money", "amount": 2000.0,
  "fee": 0.0, "sender": "Jane Smith", "receiver": "Self",
  "timestamp": "2024-05-10T14:30:58.724000+00:00"}, ...]
```

Error codes:

401 Missing or invalid credentials

2.2 GET /transactions/{id}

Returns a single transaction by its numeric ID.

Request:

```
curl -u admin:password123 http://localhost:8000/transactions/1
```

Response -- 200 OK:

```
{"id": 1, "type": "Incoming Money", "amount": 2000.0,
  "fee": 0.0, "new_balance": 2000.0, "sender": "Jane Smith",
  "receiver": "Self", "timestamp": "2024-05-10T14:30:58.724000+00:00"}
```

Error codes:

401	Missing or invalid credentials
404	Transaction not found

2.3 POST /transactions

Creates a new transaction and appends it to the dataset.

Request:

```
curl -u admin:password123 -X POST http://localhost:8000/transactions \
  -H "Content-Type: application/json" \
  -d '{"type": "Payment to Code Holder", "amount": 5000.0,
    "fee": 100.0, "sender": "Merci Ndekwe", "receiver": "MTN Rwanda",
    "timestamp": "2024-06-01T09:00:00+00:00"}'
```

Response -- 201 Created:

```
{"id": 1692, "type": "Payment to Code Holder", "amount": 5000.0,
  "fee": 100.0, "sender": "Merci Ndekwe", "receiver": "MTN Rwanda",
  "timestamp": "2024-06-01T09:00:00+00:00"}
```

Error codes:

400	Request body missing or invalid JSON
401	Missing or invalid credentials

2.4 PUT /transactions/{id}

Partially updates an existing transaction. Only supplied fields are changed.

Request:

```
curl -u admin:password123 -X PUT http://localhost:8000/transactions/1692 \
  -H "Content-Type: application/json" \
  -d '{"amount": 6000.0}'
```

Response -- 200 OK:

```
{"id": 1692, "type": "Payment to Code Holder", "amount": 6000.0,
  "fee": 100.0, "sender": "Merci Ndekwe", "receiver": "MTN Rwanda",
  "timestamp": "2024-06-01T09:00:00+00:00"}
```

Error codes:

400	Request body missing or invalid JSON
401	Missing or invalid credentials
404	Transaction not found

2.5 DELETE /transactions/{id}

Permanently removes a transaction. Returns 204 No Content with an empty body.

Request:

```
curl -u admin:password123 -X DELETE http://localhost:8000/transactions/1692
```

Response -- 204 No Content:

```
(empty body)
```

Error codes:

401	Missing or invalid credentials
404	Transaction not found

2.6 Error Response Format

All error responses return a consistent JSON body:

```
{"error": "Description of what went wrong"}
```

3. DSA Comparison -- Linear Search vs Dictionary Lookup

3.1 Methods

Linear Search -- $O(n)$: Iterates through the transaction list one record at a time until the target ID is found. In the worst case all 1,691 records are scanned.

Dictionary Lookup -- $O(1)$: At startup, transactions are loaded into a Python dictionary keyed by ID. A lookup requires only one hash computation -- no iteration.

3.2 Benchmark Results

Dataset: 1,691 transactions | Target: ID 1691 (worst case) | Runs: 500

=====			
DSA BENCHMARK: Linear Search vs Dictionary Lookup			
=====			
Dataset size : 1691 transactions			
Target ID : 1691			
Runs : 500			

Method	Avg Time (s)	Total Time (s)	

Linear Search $O(n)$	0.00005740	0.028701	
Dictionary Lookup $O(1)$	0.00000008	0.000042	

Dictionary is 683.4x faster than linear search			
=====			
Method	Avg Time per Search	Total (500 runs)	Speedup
Linear Search $O(n)$	0.00005740 s	0.028701 s	--
Dictionary Lookup $O(1)$	0.00000008 s	0.000042 s	683.4x faster

3.3 Reflection

Linear search scans every element sequentially. For 1,691 records this means up to 1,691 comparisons per query. As the dataset grows, search time grows proportionally -- $O(n)$.

Dictionary lookup uses Python's built-in hash table. The key is passed through a hash function that computes the exact memory slot where the value is stored. No iteration is required. The cost of building the dictionary is paid once at startup, not per query.

Why dictionary lookup is faster:

- Hashing converts the key directly to a memory address
- No element-by-element comparison is needed
- Lookup time is constant regardless of dataset size -- $O(1)$

Alternative data structures:

- Binary Search on sorted list -- $O(\log n)$: sort IDs once, then binary search. Faster than linear but requires maintaining sorted order on every insert.
- Binary Search Tree (BST) -- $O(\log n)$ average: supports efficient range queries in addition to point lookups, at the cost of more complex implementation.

- Hash Table with explicit collision handling -- $O(1)$ average: allows custom collision resolution strategies (chaining, open addressing) for specific workloads.

4. Basic Auth Limitations & Stronger Alternatives

4.1 What Basic Authentication Does

Basic Authentication works by encoding the username and password in Base64 and sending them in the Authorization header on every HTTP request. The server decodes the header, checks the credentials, and either allows or rejects the request with a 401 Unauthorized response.

4.2 Why Basic Auth Is Weak

Base64 is not encryption.

The credentials are only encoded, not encrypted. Anyone who intercepts the request can decode the header instantly and read the username and password in plain text.

Credentials travel on every request.

Because there is no session or token, the username and password must be attached to every single API call, increasing the attack surface and risk of credentials leaking in logs.

No expiry.

Basic Auth credentials do not expire. If compromised, the attacker has permanent access until the password is manually changed.

No scopes or roles.

Basic Auth is binary -- the user either has access or they do not. There is no way to grant limited permissions such as read-only access for one client and full access for another.

4.3 How JWT Addresses These Weaknesses

JSON Web Tokens (JWT) replace credentials with a signed token issued at login. The token contains claims (user ID, roles, expiry time) and is cryptographically signed so the server can verify it without storing session state. Tokens expire automatically, and a compromised token becomes useless after the expiry time without requiring a password change.

4.4 How OAuth2 Goes Further

OAuth2 is an authorisation framework that separates authentication from resource access. It introduces scopes, allowing clients to request only the permissions they need. It also supports token refresh flows, third-party login, and delegated access -- making it the industry standard for securing modern APIs.

5. Test Evidence & Screenshots

All endpoints were tested using tests/test_api.py against a live instance of the API server. The suite ran 11 scenarios covering all CRUD operations with both valid and invalid credentials.

#	Endpoint	Auth	Expected	Result
1	GET /transactions	Valid	200	Pass
2	GET /transactions	None	401	Pass
3	GET /transactions	Wrong creds	401	Pass
4	GET /transactions/1	Valid	200	Pass
5	GET /transactions/99999	Valid	404	Pass
6	POST /transactions	Valid	201	Pass
7	POST /transactions	None	401	Pass
8	PUT /transactions/1692	Valid	200	Pass
9	PUT /transactions/99999	Valid	404	Pass
10	DELETE /transactions/1692	Valid	204	Pass
11	DELETE /transactions/99999	Valid	404	Pass

5.1 Successful GET -- 200 OK

```
=== GET /transactions --- 200 OK ===
Status : 200
Headers: Content-Type: application/json; charset=utf-8
Body   : [{"id": 1, "type": "Incoming Money", "amount": 2000.0,
          "sender": "Jane Smith", "receiver": "Self", ...}]
```

5.2 Unauthorized -- 401 (No Credentials)

```
=== GET /transactions --- 401 Unauthorized ===
Status : 401
Headers: WWW-Authenticate: Basic realm="MoMo API"
          Content-Type: application/json; charset=utf-8
Body   : {"error": "unauthorized"}
```

5.3 Unauthorized -- 401 (Wrong Credentials)

```
=== GET /transactions --- wrong creds 401 ===
Status : 401
Headers: WWW-Authenticate: Basic realm="MoMo API"
          Content-Type: application/json; charset=utf-8
Body   : {"error": "unauthorized"}
```

5.4 Successful POST -- 201 Created

```
=== POST /transactions --- 201 Created ===
Status : 201
```

```
Headers: Content-Type: application/json; charset=utf-8
        Content-Length: 172
Body    : {"type": "Payment to Code Holder", "amount": 5000.0,
          "fee": 100.0, "sender": "Merci Ndekwe",
          "receiver": "MTN Rwanda", "id": 1692}
```

5.5 Successful PUT -- 200 OK

```
=== PUT /transactions/1692 --- 200 OK ===
Status : 200
Headers: Content-Type: application/json; charset=utf-8
        Content-Length: 172
Body    : {"type": "Payment to Code Holder", "amount": 6000.0,
          "fee": 100.0, "sender": "Merci Ndekwe",
          "receiver": "MTN Rwanda", "id": 1692}
```

5.6 Successful DELETE -- 204 No Content

```
=== DELETE /transactions/1692 --- 204 No Content ===
Status : 204
Headers: (no Content-Type -- empty body)
Body    : (empty)
```

6. Conclusion

This project demonstrated the full lifecycle of a secured REST API: parsing raw XML data, exposing it through five CRUD endpoints, protecting those endpoints with authentication, documenting the interface, and comparing search algorithm efficiency.

The DSA benchmark confirmed that dictionary-based lookups ($O(1)$) are 683x faster than linear search ($O(n)$) on the 1,691-record dataset, validating the architectural decision to maintain an in-memory ID index in the server at startup.

While Basic Authentication served as a functional learning exercise, production deployment would require migration to JWT or OAuth2 to address its fundamental security limitations -- namely, credentials exposed on every request, no expiry mechanism, and no support for granular permissions or scopes.